

EC801 Logic Synthesis Assignment 1

Team members:

1. Rohit L (231EC248)
2. Pranav Jois (231EC237)
3. Manal Ahmed (231EC135)

➤ **Introduction:**

This assignment implements the core Boolean algebra operations including cofactor extraction, boolean difference, consensus, and smoothing for expressions given in Sum of Products (SOP) form.

The implementation is done in one Python file: **231EC237_231EC248_231EC135_Code1.py**

The main function handles the input interface: it reads the boolean expression from the specified text file and collects the expression number and control variable/variables from the user.

The Boolean functions are defined in the beginning which implement the underlying operations that the main function uses: cofactor computation, boolean difference, consensus, and smoothing.

Our code can be accessed here: <https://github.com/RohitLalu/EC801-Logic-Synthesis-Coursework>

➤ **Naming Convention:**

Normal boolean variable: Lowercase letter

Complement form: Uppercase letter

Example: a represents "a" and A represents $\bar{a}$

$$aB + Ab = a\bar{b} + \bar{a}b$$

➤ **Input Format:**

The boolean expressions are stored in a text file with one expression in each line. Minterms are separated using + operator.

Example input provided in our test.txt file:

a + Ab

aB + Ab

ABC + Abc + bC + aBc

ABC + Abc + bC + aBc + ab

The program will ask for the name of the input file and the expression number (line number) which has to be evaluated.

Then the user has to enter if there is one control variable or multiple.

Example: a, ab, aB, Ab

The variables should be entered without any spaces.

➤ **Methodology:**

The expression is read from the text file and split into its individual product terms, which are stored in a list.

Control variable: The variable with respect to which the boolean operation is performed. It can be a single variable (a/ b/ c) or multi variable (aB/ Ab/ ab/ AB).

Example: if the control variable is a, then the program will find the cofactors wrt a i.e. f_a and $f_{a'}$. If the control variable is ab, then it finds f_{ab}

The cofactor is found by checking if the variable is present in the product terms, if yes then print it without that variable, if not print it as it is. This method is chosen as the input is taken in string format.

The multi variable cofactor is found using recursion. First find f_a then take input as f_a and find its cofactor wrt b. That will give f_{ab}

Shannon Expansion: $F = x f_x + x' f_{x'}$ This is used to find the function using its cofactors.

Cofactor: Cofactor of f wrt x is $f(x=1)$ and wrt x' is $f(x=0)$

Boolean Difference: With respect to x, it is $f_x \oplus f_{x'}$

It is the XOR function of the cofactors of f with respect to x.

Consensus: It is an AND operation between the cofactors of f wrt x.

Mathematically, it is: $f_x \cdot f_{x'}$

Smoothing: It is the OR operation between the cofactors of f wrt x.

Mathematically, it is: $f_x + f_{x'}$

➤ **Steps to execute the program:**

1. Place **231EC237_231EC248_231EC135_Code1.py** and the **input test file** (.txt file) in the same directory.
2. Open terminal in that directory.
3. Run the python program using - `python 231EC237_231EC248_231EC135_Code1.py`
4. Enter the name of the input test file. (*example: test.txt*)
5. Enter the expression number (line number) which has to be evaluated.
6. Select whether there are single (1) or multiple (2) control variables.
7. Enter the control variables. For 2 variables, enter without space in between the variables.
8. The program displays the cofactors, boolean difference, consensus and smoothing function.

➤ **Screenshots of execution of a boolean expression:**

This is the input sample test file which includes 5 boolean expressions:

```

a + Ab
aB + Ab|
ABC + Abc + bC + aBc
ABC + Abc + bC + aBc + ab
a+ bc + DE + FGH + Af + Ijk + lMN + bn + dhk + eIn

```

Python program output for expression 4: $\bar{a} \bar{b} \bar{c} + \bar{a} b c + b \bar{c} + a b \bar{c} + ab$

Single control variable: b

$f_{\bar{b}} = \bar{a} \bar{c} + \bar{c} + a$

$f_{b} = \bar{a} \bar{c} + a c$

Boolean difference = $\bar{a} \bar{c} + a \bar{c}$

Consensus = $\bar{a} \bar{c} + a c$

Smoothing = 1

```

PS C:\Users\ASUS\OneDrive\Documents\NITK\7th Sem\Logic Synthesis\Assignment 1\q1> python 231EC237_231EC248_231EC135_Code1.py
Enter test file name: test.txt
Enter which expression to eval: 4
Do you want to enter
1. control variable?
2. control variables?
Enter 1 or 2: 1
Enter control variable: b
f_var:Ac + C + a

f_var_bar:AC + ac

f_bool_diff:Ac + aC

f_consensus:AC + ac

f_smoothing:1

```

Python program output for expression 4: $\bar{a} \bar{b} \bar{c} + \bar{a} b c + b \bar{c} + a b \bar{c} + ab$

2 control variables: b $\bar{a}$

$f_{b \bar{a}} = c + \bar{c}$

Boolean difference = $\bar{b} \bar{a} c + \bar{b} a \bar{c} + b \bar{a} \bar{c} + b a c$

Consensus = 0

Smoothing = 1

```

PS C:\Users\ASUS\OneDrive\Documents\NITK\7th Sem\Logic Synthesis\Assignment 1\q1> python 231EC237_231EC248_231EC135_Code1.py
Enter test file name: test.txt
Enter which expression to eval: 4
Do you want to enter
1. control variable?
2. control variables?
Enter 1 or 2: 2
Enter control variables (together no spaces in the order to be evaluated): bA
f_bA:c + C

f = f_bA(c + C) + f_BA(C) + f_Ba(c) + f_ba(1)
f_bool_diff_bA:BAc + BaC + bAC + bac

f_consensus_bA:0

f_smoothing_bA:1

```

➤ Conclusion:

The program correctly computes the cofactors of a Sum of Products (SOP) expression with respect to both single and multiple control variables, and derives the corresponding boolean difference, consensus, and smoothing functions through recursive cofactor extraction.

Because expressions are read from an external text file, additional boolean expressions can be evaluated without modifying the code, and the recursive cofactor routine extends naturally to any number of control variables.

➤ Resources:

1. <https://www.flyriver.com/g/espresso-heuristic-logic-minimizer?auth=1788630669978>
2. <https://www.flyriver.com/g/optimizing-boolean-expression?auth=1788630617343>
3. https://en.wikipedia.org/wiki/Quine–McCluskey_algorithm
4. <https://stackoverflow.com/questions/11324804/algorithm-minimizing-boolean-expressions>
5. <https://stackoverflow.com/questions/464864/how-to-get-all-possible-2n-combinations-of-a-list-s-elements-of-any-length>
6. <https://math.stackexchange.com/questions/505393/how-many-semantically-different-boolean-functions-are-there-for-n-boolean-variab>